Laboratorio 2: Métodos supervisados de clasificación

Isaac Sibaja Cortes – 402620823
Josué Peñaranda Alvarado – 119750839
Luis Diego Alvarado Salazar – 119290953

EIF420O – Inteligencia Artificial
Universidad Nacional de Costa Rica

Resumen

Este laboratorio extiende el framework de análisis de datos desarrollado en el curso con una clase de aprendizaje supervisado para clasificación. Se implementaron y compararon los algoritmos K Vecinos Más Cercanos (KNN), Árbol de Decisión (DT), Random Forest (RF), XGBoost y AdaBoost sobre dos conjuntos de datos: *Pima Indians Diabetes* y *Water Potability*. Para cada algoritmo se evaluó una configuración estándar y varias variaciones de hiperparámetros, se seleccionó el mejor modelo y se analizó su desempeño mediante *accuracy*, precisión, *recall*, F1 macro y matriz de confusión. Los métodos de ensamble basados en árboles obtuvieron los mejores resultados en ambos conjuntos de datos.

1. Introducción

En el aprendizaje supervisado, cada observación del conjunto de datos contiene un conjunto de atributos de entrada X y una respuesta conocida y. El objetivo es aprender una función f tal que $f(X) \approx y$, para luego predecir la respuesta de observaciones nuevas. Cuando y toma valores dentro de un conjunto finito de categorías, el problema se denomina **clasificación** [7].

En este laboratorio se integró a la clase de métodos supervisados del framework (`ClasificacionModelos`, que hereda de `SupervisadoBase`) un conjunto de algoritmos de clasificación, y se realizaron experimentos reproducibles con dos problemas de clasificación binaria: predecir si una paciente tiene diabetes y predecir si una muestra de agua es potable.

2. Marco teórico

2.1. Mapa conceptual

La Figura 1 resume los algoritmos estudiados según la estrategia que utilizan para clasificar, junto con los elementos del proceso de evaluación.

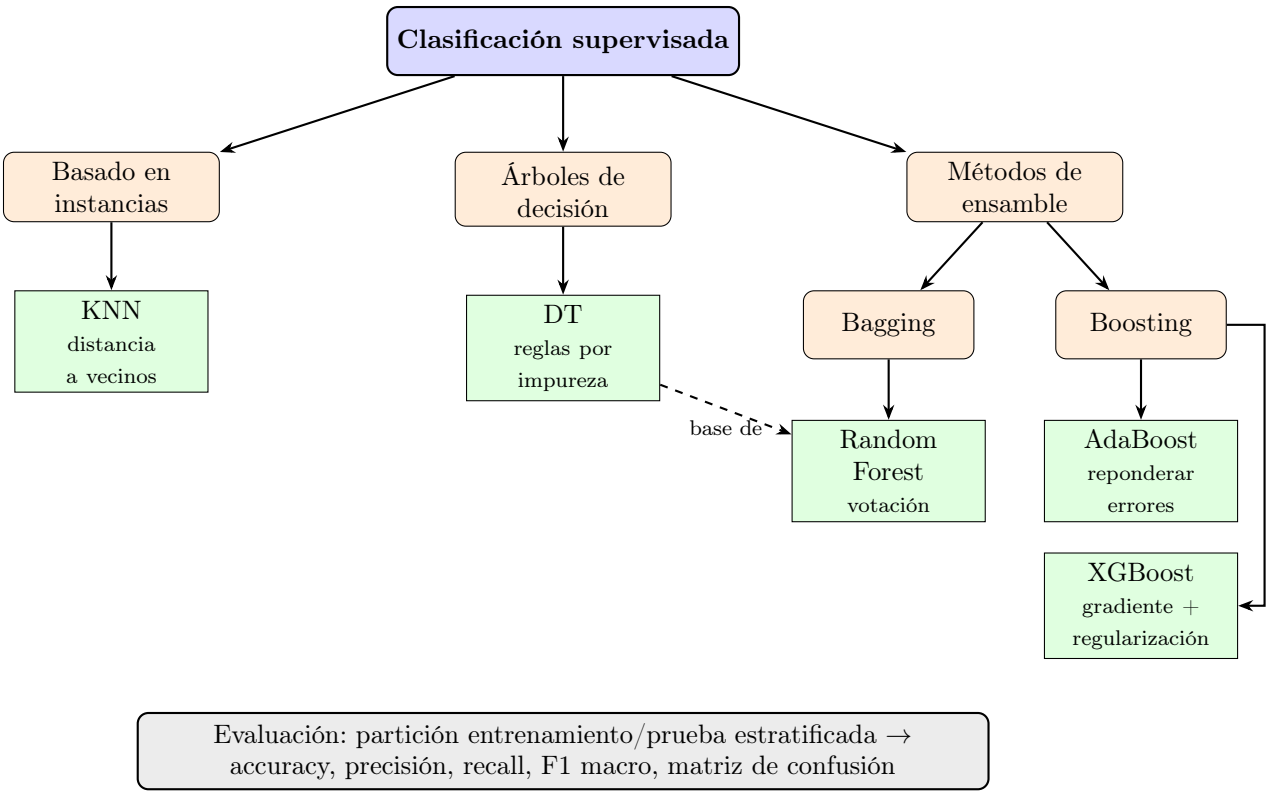


Figura 1: Mapa conceptual de los algoritmos de clasificación estudiados.

2.2. K Vecinos Más Cercanos (KNN)

KNN es un método basado en instancias: no construye un modelo explícito durante el entrenamiento, sino que almacena las observaciones y, para clasificar un punto nuevo, busca las k observaciones más cercanas y asigna la clase más frecuente entre ellas [4]. La cercanía se mide habitualmente con la distancia euclidiana:

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{j=1}^{p} (x_j - x'_j)^2}.$$

Debido a que depende de distancias, KNN es sensible a la escala de las variables, por lo que es necesario estandarizarlas. El hiperparámetro principal es k: valores pequeños producen fronteras de decisión muy irregulares y sensibles al ruido, mientras que valores grandes suavizan la frontera y favorecen a la clase mayoritaria.

2.3. Árboles de Decisión (DT)

Un árbol de decisión divide recursivamente el espacio de atributos mediante reglas de la forma "$x_j \leq t$". En cada nodo se elige la variable y el umbral que más reducen la impureza de los nodos hijos [1, 10]. En la implementación utilizada se emplea el índice de Gini:

$$G = 1 - \sum_{c=1}^{C} p_c^2,$$

donde p_c es la proporción de observaciones de la clase c en el nodo. Los árboles son fáciles de interpretar, pero un árbol sin restricciones tiende a sobreajustarse. Para controlarlo se limitan hiperparámetros como la profundidad máxima (`max_depth`) y el número mínimo de muestras para dividir un nodo (`min_samples_split`).

2.4. Random Forest (RF)

Random Forest es un método de ensamble que entrena muchos árboles de decisión, cada uno sobre una muestra *bootstrap* del conjunto de entrenamiento y considerando en cada división solo un subconjunto aleatorio de variables [2]. La clase final se obtiene por votación mayoritaria. La aleatoriedad hace que los árboles sean poco correlacionados entre sí, de modo que al promediarlos se reduce la varianza respecto a un árbol individual. Sus hiperparámetros principales son el número de árboles (`n_estimators`) y los que controlan el tamaño de cada árbol.

2.5. AdaBoost

AdaBoost (*Adaptive Boosting*) construye una secuencia de clasificadores débiles, típicamente árboles de un solo nivel [5]. Inicialmente todas las observaciones tienen el mismo peso; después de cada iteración, se aumenta el peso de las observaciones mal clasificadas, de modo que el siguiente clasificador se enfoque en ellas. La predicción final es una votación ponderada:

$$F(\mathbf{x}) = \operatorname{sign}\left(\sum_{m=1}^{M} \alpha_m h_m(\mathbf{x})\right),$$

donde α_m refleja la precisión del clasificador h_m. Sus hiperparámetros son el número de estimadores M y la tasa de aprendizaje. Es sensible a valores atípicos y a etiquetas ruidosas, ya que estas reciben pesos cada vez mayores.

2.6. XGBoost

XGBoost (*Extreme Gradient Boosting*) es una implementación eficiente del *gradient boosting* de árboles [6, 3]. Los árboles se agregan secuencialmente y cada uno se ajusta para corregir el error residual del ensamble actual, siguiendo la dirección del gradiente de la función de pérdida. A diferencia del *gradient boosting* clásico, XGBoost minimiza una función objetivo que incluye un término de regularización:

$$\mathcal{L} = \sum_i \ell(y_i, \hat{y}_i) + \sum_m \Omega(f_m), \qquad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda \|w\|^2,$$

donde T es el número de hojas del árbol y w los valores de sus hojas. Los hiperparámetros principales son el número de árboles, la profundidad máxima y la tasa de aprendizaje (`learning_rate`), que escala la contribución de cada árbol.

2.7. Métricas de evaluación

Para un problema binario, la matriz de confusión registra verdaderos positivos (VP), falsos positivos (FP), verdaderos negativos (VN) y falsos negativos (FN). A partir de ella se definen [12]:

$$\text{Accuracy} = \frac{VP + VN}{VP + VN + FP + FN}, \qquad \text{Precisión} = \frac{VP}{VP + FP},$$

$$\text{Recall} = \frac{VP}{VP + FN}, \qquad F_1 = 2 \cdot \frac{\text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}}.$$

El promedio **macro** calcula cada métrica por clase y luego las promedia, dando el mismo peso a ambas clases. Por esta razón, se utilizó el **F1 macro** como métrica principal, ya que la *accuracy* puede ser engañosa cuando las clases están desbalanceadas.

3. Metodología

3.1. Implementación en el framework

Los algoritmos se integraron en la clase `ClasificacionModelos` del paquete `src/supervised`, utilizando las implementaciones de `scikit-learn` [9] y de la librería `xgboost` [3]. Cada método (`knn`, `decision_tree`, `random_forest`, `xgboost`, `adaboost`) recibe sus hiperparámetros y devuelve el modelo entrenado junto con sus métricas. Adicionalmente, `adaboost_grid` realiza una búsqueda en cuadrícula con validación cruzada.

El preprocesamiento forma parte de un `Pipeline`, de modo que se ajusta únicamente con los datos de entrenamiento y se evita filtrar información del conjunto de prueba. Las variables numéricas se imputan con la mediana y se estandarizan.

Los experimentos se encuentran en el cuaderno `notebooks/Experimentacion_Clasificacion.ipynb` y son reproducibles. La corrección del código se verifica con las pruebas automáticas de `tests/test_mvc.py` (22 pruebas exitosas).

3.2. Diseño experimental

- **Partición:** 75 % entrenamiento y 25 % prueba, estratificada y con semilla fija (`random_state = 42`). Todos los modelos de un mismo conjunto de datos usan exactamente la misma partición.
- **Línea base:** *accuracy* obtenida al predecir siempre la clase mayoritaria. Un modelo útil debe superarla.
- **Variaciones:** para cada algoritmo se evaluó la configuración estándar del framework y entre 3 y 5 variaciones, cambiando un hiperparámetro a la vez (Tabla 1).
- **Selección:** se eligió la configuración con mayor F1 macro en el conjunto de prueba.

Algoritmo	Configuración estándar	Variaciones evaluadas
KNN	$k = 5$	$k \in \{3, 7, 11, 15, 21\}$
DT	sin límite de profundidad	`max_depth` $\in \{3, 5, 8\}$, `min_samples_split`= 20
RF	200 árboles	100 y 500 árboles, `max_depth`= 8, `min_samples_split`= 10
XGBoost	120 árboles, prof. 3, lr 0.1	lr 0.05 (300 árboles), lr 0.3, prof. 2 y 5
AdaBoost	80 estimadores	50, 150 y 300 estimadores; búsqueda en cuadrícula

Tabla 1: Configuraciones evaluadas por algoritmo.

4. Conjuntos de datos

4.1. Diabetes

El conjunto *Pima Indians Diabetes* [11] contiene 768 registros de mujeres de origen pima de al menos 21 años, con 8 variables clínicas: número de embarazos, glucosa, presión arterial, grosor del pliegue cutáneo, insulina, índice de masa corporal, función de pedigrí de diabetes y edad. La variable

objetivo `Outcome` indica si la paciente tiene diabetes (1) o no (0). Las clases están desbalanceadas: 500 negativas (65.1 %) y 268 positivas (34.9 %).

Preprocesamiento: el conjunto no contiene valores nulos explícitos, pero utiliza el valor 0 para representar datos faltantes en variables donde es fisiológicamente imposible: `Insulin` (374 ceros), `SkinThickness` (227), `BloodPressure` (35), `BMI` (11) y `Glucose` (5). Estos ceros se convirtieron a valores nulos para que el pipeline los imputara con la mediana, en lugar de tratarlos como mediciones reales.

4.2. Potabilidad del agua

El conjunto *Water Potability* [8] contiene 3276 muestras de agua con 9 mediciones fisicoquímicas: pH, dureza, sólidos disueltos, cloraminas, sulfatos, conductividad, carbono orgánico, trihalometanos y turbidez. La variable objetivo `Potability` indica si el agua es apta para consumo humano (1) o no (0). La distribución es de 1998 muestras no potables (61.0 %) y 1278 potables (39.0 %).

Preprocesamiento: el conjunto presenta valores nulos en `ph` (491), `Sulfate` (781) y `Trihalomethanes` (162). No se eliminaron filas para no perder información; los nulos se imputaron con la mediana calculada sobre el conjunto de entrenamiento.

5. Resultados y análisis: Diabetes

La partición produjo 576 registros de entrenamiento y 192 de prueba. La línea base es de 0.651.

Algoritmo	Configuración	Accuracy	Precisión	Recall	F1 macro
KNN	$k = 5$ (estándar)	0.7344	0.7070	0.6921	**0.6975**
KNN	$k = 21$	0.7396	0.7151	0.6892	0.6970
KNN	$k = 11$	0.7344	0.7075	0.6887	0.6950
KNN	$k = 3$	0.7135	0.6836	0.6796	0.6814
DT	`max_depth`= 5	0.7500	0.7247	0.7214	**0.7229**
DT	`max_depth`= 5, `min_split`= 20	0.7396	0.7128	0.7065	0.7093
DT	`max_depth`= 3	0.7083	0.6763	0.6513	0.6574
DT	sin límite (estándar)	0.6875	0.6496	0.6284	0.6330
RF	100 árboles	0.7500	0.7259	0.7076	**0.7141**
RF	`min_split`= 10	0.7448	0.7189	0.7070	0.7118
RF	200 árboles (estándar)	0.7448	0.7201	0.7001	0.7069
RF	500 árboles	0.7396	0.7135	0.6961	0.7022
AdaBoost	150 estimadores	0.7448	0.7189	0.7070	**0.7118**
AdaBoost	80 estimadores (estándar)	0.7396	0.7135	0.6961	0.7022
AdaBoost	300 estimadores	0.7240	0.6949	0.6772	0.6830

Tabla 2: Diabetes: resultados de las principales configuraciones por algoritmo.

KNN. La mejor configuración fue la estándar ($k = 5$). Con $k = 3$ el desempeño disminuyó por la sensibilidad al ruido, y con valores mayores de k la *accuracy* se mantuvo pero el F1 no mejoró, ya que la clase mayoritaria tiende a dominar la votación. KNN detecta solo el 55 % de las pacientes con diabetes (*recall* de la clase 1 = 0.55).

Árbol de Decisión. El árbol sin restricciones obtuvo el peor resultado (F1 = 0.633), lo que evidencia sobreajuste. Limitar la profundidad a 5 elevó el F1 a 0.723, mientras que con profundidad

3 el modelo quedó demasiado simple (F1 = 0.657). La profundidad 5 representa el mejor balance entre complejidad y generalización, y fue el modelo con mayor *recall* en la clase positiva (0.63).

Random Forest. Todas las configuraciones obtuvieron resultados muy similares (F1 entre 0.702 y 0.714), lo que muestra su estabilidad ante cambios de hiperparámetros. Aumentar a 500 árboles no aportó mejora. Comparado con el árbol sin restricciones, Random Forest mejora claramente gracias a la reducción de varianza del ensamble.

XGBoost. La mejor configuración fue `learning_rate`= 0,05 con 300 árboles (F1 = 0.726, *accuracy* = 0.755), que fue el mejor modelo del conjunto de datos. Una tasa de aprendizaje menor hace que cada árbol corrija los errores de forma gradual, y al compensarla con más árboles se obtiene un modelo que generaliza mejor.

AdaBoost. La mejor variación fue con 150 estimadores (F1 = 0.712); con 300 el desempeño bajó, lo que sugiere que demasiadas iteraciones concentran el modelo en observaciones ruidosas. La búsqueda en cuadrícula seleccionó `learning_rate`= 0,1 y 100 estimadores (*accuracy* en validación cruzada de 0.776), pero en prueba obtuvo F1 = 0.672, inferior a la variación manual, porque la búsqueda optimiza la *accuracy* y no el F1.

Algoritmo	Mejor configuración	Accuracy	Precisión	Recall	F1 macro
XGBoost	lr= 0,05, 300 árboles	**0.7552**	0.7306	0.7220	**0.7257**
Árbol de Decisión	`max_depth`= 5	0.7500	0.7247	0.7214	0.7229
Random Forest	100 árboles	0.7500	0.7259	0.7076	0.7141
AdaBoost	150 estimadores	0.7448	0.7189	0.7070	0.7118
KNN	$k = 5$	0.7344	0.7070	0.6921	0.6975
Línea base (clase mayoritaria)		0.6510	–	–	–

Tabla 3: Diabetes: comparación de los mejores modelos de cada algoritmo.

Análisis comparativo. El mejor modelo fue XGBoost, aunque la diferencia con el Árbol de Decisión (max_depth = 5) es de apenas 0.003 en F1, por lo que en la práctica ambos rinden casi igual; el árbol tiene la ventaja de ser interpretable. Todos los modelos superan la línea base por entre 8 y 10 puntos de *accuracy*. En todos los casos el F1 macro es menor que la *accuracy*, lo que indica que la clase minoritaria (pacientes con diabetes) es más difícil de detectar. En un contexto médico este es el error más grave, pues un falso negativo implica no diagnosticar a una persona que sí tiene la enfermedad.

6. Resultados y análisis: Potabilidad del agua

La partición produjo 2457 registros de entrenamiento y 819 de prueba. La línea base es de 0.609.

Algoritmo	Configuración	Accuracy	Precisión	Recall	F1 macro
KNN	$k = 3$	0.6081	0.5773	0.5691	**0.5685**
KNN	$k = 5$ (estándar)	0.6093	0.5742	0.5605	0.5563
KNN	$k = 15$	0.6410	0.6233	0.5709	0.5521
KNN	$k = 21$	0.6398	0.6275	0.5637	0.5369
DT	`min_split`= 20	0.5849	0.5683	0.5696	**0.5686**
DT	sin límite (estándar)	0.5592	0.5426	0.5436	0.5426
DT	`max_depth`= 8	0.6190	0.5858	0.5366	0.4957
DT	`max_depth`= 3	0.6215	0.6557	0.5201	0.4321
RF	`min_split`= 10	0.6728	0.6721	0.6093	**0.6012**
RF	500 árboles	0.6740	0.6755	0.6097	0.6012
RF	200 árboles (estándar)	0.6728	0.6730	0.6087	0.6002
RF	`max_depth`= 8	0.6581	0.6962	0.5743	0.5385
AdaBoost	300 estimadores	0.6190	0.5949	0.5271	**0.4655**
AdaBoost	150 estimadores	0.6178	0.5975	0.5221	0.4504
AdaBoost	80 estimadores (estándar)	0.6166	0.5938	0.5200	0.4454

Tabla 4: Potabilidad: resultados de las principales configuraciones por algoritmo.

KNN. El mejor resultado se obtuvo con $k = 3$ (F1 = 0.569), aunque su *accuracy* es prácticamente igual a la línea base. Al aumentar k la *accuracy* sube pero el F1 baja, porque el modelo predice cada vez más la clase mayoritaria. Esto indica que las muestras potables y no potables están muy mezcladas en el espacio de variables.

Árbol de Decisión. La mejor configuración fue `min_samples_split`= 20 (F1 = 0.569). A diferencia de Diabetes, limitar la profundidad empeoró el F1 (hasta 0.432 con profundidad 3), ya que un árbol poco profundo termina prediciendo casi siempre "no potable". Exigir un mínimo de muestras por división reduce el sobreajuste sin eliminar la capacidad de identificar agua potable.

Random Forest. Fue el algoritmo más estable y el mejor del conjunto de datos (F1 = 0.601). Limitar la profundidad redujo el F1 a 0.539, lo que indica que el problema requiere árboles complejos para capturar relaciones no lineales entre las variables químicas. Su principal limitación es que solo detecta el 32 % del agua potable (*recall* de la clase 1 = 0.32).

XGBoost. La mejor configuración fue `max_depth`= 5 (F1 = 0.594), muy cerca de Random Forest. Que la configuración de mayor profundidad sea la mejor coincide con lo observado en Random Forest: las relaciones entre las variables y la potabilidad no son simples.

AdaBoost. Fue el peor modelo. Su *accuracy* (0.619) apenas supera la línea base, pero su F1 macro es de solo 0.466, ya que detecta únicamente el 11 % del agua potable. La búsqueda en cuadrícula (`learning_rate`= 1,0, 200 estimadores) obtuvo un resultado similar (F1 = 0.460). Los clasificadores débiles de AdaBoost no logran separar las clases cuando ninguna variable individual es buena predictora.

Algoritmo	Mejor configuración	Accuracy	Precisión	Recall	F1 macro
Random Forest	`min_split`= 10	**0.6728**	0.6721	0.6093	**0.6012**
XGBoost	`max_depth`= 5	0.6581	0.6424	0.6001	0.5942
Árbol de Decisión	`min_split`= 20	0.5849	0.5683	0.5696	0.5686
KNN	$k = 3$	0.6081	0.5773	0.5691	0.5685
AdaBoost	300 estimadores	0.6190	0.5949	0.5271	0.4655
Línea base (clase mayoritaria)		0.6093	–	–	–

Tabla 5: Potabilidad: comparación de los mejores modelos de cada algoritmo.

Análisis comparativo. El mejor modelo fue Random Forest, seguido de cerca por XGBoost. Solo estos dos superan claramente la línea base; KNN y el Árbol de Decisión quedan prácticamente a su nivel, y AdaBoost la supera en *accuracy* pero con un F1 muy bajo. Este conjunto de datos resultó considerablemente más difícil que el de Diabetes: las variables fisicoquímicas tienen una relación débil con la potabilidad y las clases se traslapan. En la práctica, el error más peligroso es clasificar como potable un agua que no lo es; el modelo ganador acierta el 67 % de las veces que predice "potable", por lo que un 33 % de esas predicciones corresponderían a agua no apta para consumo.

7. Conclusiones

- Los métodos de ensamble basados en árboles (XGBoost y Random Forest) fueron los más consistentes, ocupando los primeros lugares en ambos conjuntos de datos, gracias a que reducen la varianza (Random Forest) o corrigen errores de forma secuencial (XGBoost).
- En Diabetes, un Árbol de Decisión bien regularizado alcanzó casi el mismo desempeño que XGBoost, con la ventaja de ser interpretable. Esto muestra la importancia de ajustar los hiperparámetros: el mismo algoritmo pasó de F1 = 0.633 a 0.723 solo limitando su profundidad.
- El conjunto de Diabetes fue más fácil de clasificar (F1 máximo de 0.726) que el de Potabilidad (F1 máximo de 0.601), donde varios modelos apenas superaron la línea base.
- La *accuracy* no es suficiente cuando las clases están desbalanceadas: AdaBoost en Potabilidad obtuvo una *accuracy* aceptable pero casi no detectaba agua potable. Por ello se utilizó el F1 macro como métrica principal.
- Como limitación, los resultados provienen de una sola partición entrenamiento/prueba, por lo que diferencias pequeñas no son concluyentes. Como trabajo futuro se propone aplicar validación cruzada a todos los algoritmos y técnicas de balanceo de clases.

Referencias

[1] L. Breiman, J. H. Friedman, R. A. Olshen y C. J. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth, 1984.

[2] L. Breiman, "Random forests", *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[3] T. Chen y C. Guestrin, "XGBoost: A scalable tree boosting system", en *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[4] T. Cover y P. Hart, "Nearest neighbor pattern classification", *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[5] Y. Freund y R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting", *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[6] J. H. Friedman, "Greedy function approximation: A gradient boosting machine", *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[7] T. Hastie, R. Tibshirani y J. Friedman, *The Elements of Statistical Learning*, 2.ª ed. New York: Springer, 2009.

[8] A. Kadiwal, "Water Quality", Kaggle, 2021. [En línea]. Disponible: `https://www.kaggle.com/datasets/adityakadiwal/water-potability`

[9] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python", *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[10] J. R. Quinlan, "Induction of decision trees", *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.

[11] J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler y R. S. Johannes, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus", en *Proc. Annual Symposium on Computer Application in Medical Care*, 1988, pp. 261–265.

[12] M. Sokolova y G. Lapalme, "A systematic analysis of performance measures for classification tasks", *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.